Project 1 Report: Linux Scheduling and Multicore Performance

Purpose and Research Questions

This project investigates how Linux processor affinity, the number of eligible logical CPUs, worker-thread count, and oversubscription affect a controlled multithreaded workload.

The main project question is How do CPU affinity, eligible CPU count, thread count, and oversubscription affect the observed placement and performance of a controlled multithreaded Linux workload?

One of my extension questions I listed was How does elapsed time change as the number of worker threads increases from 1 to 8 when all workers are restricted to the same two eligible CPUs?

Scheduling placement matters because the operating system decides where runnable threads execute. Processor affinity limits the CPUs available to a process, while unrestricted execution gives the scheduler a larger placement set. Multicore performance depends on available execution capacity, worker competition, and migration between CPUs.

Prediction

Before the final trials, I predicted that unrestricted execution would be the fastest because four workers could just use the full CPU set available to the process. I also predicted that one-CPU affinity would be slowest among the four-worker configurations because all four workers would share one logical CPU. Finally, I predicted that two-CPU affinity would be faster than one-CPU affinity but slower than unrestricted execution. I also predicted that eight workers on two CPUs would take longer than four workers on two CPUs because more workers would compete for the same two CPUs. I expected one-CPU traces to report only CPU 0, two-CPU traces to report only CPUs 0 and 1, and unrestricted traces to show a wider placement set. For the extension, I predicted that increasing the thread count would eventually provide diminishing returns and then become slower as more workers competed for the same two CPUs.

Linux Environment

The experiment was run on my main laptop running Arch Linux. At the time of running the experiment, the version of the kernel was Linux `7.2.7-arch1-1`. The CPU on this laptop is an AMD Ryzen 5 7640U. The processor has 12 logical CPUS. 6 physical cores and 2 hardware threads per core. This CPU therefore allows for the following configuration:

Configuration	Eligible CPU set
Unrestricted	Inherited set, CPUs 0-11

Configuration	Eligible CPU set
One CPU	CPU 0
Two CPUs	CPUs 0 and 1
Oversubscribed	8 workers on CPUs 0 and 1
Extension	1, 2, 4, and 8 workers on CPUs 0 and 1

At the time of running the experiment, the version of GCC that was installed was GCC 16.2.1. The command used to build the C program was

```
gcc -O2 -Wall -Wextra -pthread sched_probe.c -o sched_probe
```

This CPU has the ability to automatically boost its clock speed from its base frequency, so this adds a possibility of producing timing variations.

Program and Script Design

sched_probe.c

The program accepts THREADS, SAMPLES, and WORK_UNITS. It allocates one pthread handle and one argument structure per worker. Each worker samples its current logical CPU with `sched_getcpu()`. prints the thread, sample, and CPU, and then performs the configured deterministic CPU-bound arithmetic workload.

A mutex protects output so that multiple workers do not interleave partial lines. The main thread measures elapsed time with `clock_gettime(CLOCK_MONOTONIC)` joins every worker, combines the per-worker results into a correctness checksum, and prints a final summary line.

run_affinity_trials.sh

The Bash script validates its three arguments, creates the traces directory, removes old trace files, initializes timing.csv, and runs three trials for each required configuration. The configurations are unrestricted execution, four workers on one CPU, four workers on two CPUs, and eight workers on two CPUs. The script applies fixed affinity with `taskset -c`, records elapsed, user, and system CPU time with /usr/bin/time, and saves each trace under a unique filename.

The extension compares 1, 2, 4, and 8 workers on CPUs 0 and 1. These changes extend the Week 4 placement probe with CPU-bound work, internal timing, correctness output, oversubscription, and automated repeated trials.

Experimental Methodology

The common matrix used four workers and 12 CPU-placement samples per worker. Each condition used three trials and 50,000,000 work units per worker per sample. The extension changed thread count while keeping the two-CPU affinity set, sample count, workload size, and trial count constant.

Independent Variables

1. CPU affinity configuration and number of eligible logical CPUs
2. Number of worker threads
3. For the extension, thread count: 1, 2, 4, and 8

Dependent Variables

1. User CPU time
2. System CPU time
3. Elapsed time
4. Logical CPU placement from sched_getcpu() samples
5. Worker migration between sampled CPUs

Controlled Variables

1. Operating environment and hardware
2. Sample count: 12 samples per worker
3. Workload size: 5,000,000 work units
4. Three trials per configuration
5. Same CPU affinity sets for corresponding configurations

Common-Configuration Results

The following values were calculated from the three raw rows per condition in `timing.csv`.

Configuration	Mean	Median	Minimum	Maximum	Range
Unrestricted	0.283 s	0.280 s	0.280 s	0.290 s	0.010 s
One CPU	1.133 s	1.130 s	1.120 s	1.150 s	0.030 s
Two CPUs	0.783 s	0.780 s	0.770 s	0.800 s	0.030 s
Oversubscribed	1.543 s	1.540 s	1.540 s	1.550 s	0.010 s

The unrestricted configuration had the lowest mean elapsed time at 0.283 seconds. One-CPU affinity averaged 1.133 seconds, approximately four times the unrestricted mean. Two-CPU affinity averaged 0.783 seconds, faster than one CPU but slower than unrestricted execution. Oversubscription was slowest at 1.543 seconds.

Student-Designed Extension

The extension compared 1, 2, 4, and 8 workers on CPUs 0 and 1. Every condition used 12 samples, 50,000,000 work units, and three trials.

Threads	Mean elapsed	Median	Minimum	Maximum	Observed migrations
1	0.280 s	0.280 s	0.280 s	0.280 s	1
2	0.380 s	0.380 s	0.380 s	0.380 s	0
4	0.770 s	0.770 s	0.770 s	0.770 s	5
8	1.553 s	1.550 s	1.550 s	1.560 s	9

The extension did not show a speedup from adding workers. Mean elapsed time increased from 0.280 seconds with one worker to 1.553 seconds with eight workers. Because each worker performed the configured workload, increasing thread count also increased total work. Therefore, this result measures worker-count and total-work growth together rather than fixed-total-work scalability.

Evidence-Based Analysis

Affinity affected performance

The median elapsed times were 0.280 seconds unrestricted, 1.130 seconds on one CPU, and 0.780 seconds on two CPUs. Four-worker execution took substantially longer when restricted to one CPU, while adding a second eligible CPU reduced the completion time. This is consistent with reduced parallel capacity. Four CPU-bound workers sharing one logical CPU must take turns, while two eligible CPUs provide more simultaneous execution capacity.

Oversubscription increased elapsed time

The four-worker, two-CPU condition had a median of 0.780 seconds. The eight-worker, two-CPU condition had a median of 1.540 seconds. The oversubscribed condition was approximately 1.97 times slower. Eight runnable workers competed for two eligible CPUs. The scheduler can time-share those workers, but time-sharing does not create additional execution capacity.

Affinity restrictions were applied

One-CPU traces reported only CPU 0. Two-CPU and oversubscribed traces reported only CPUs 0 and 1. Unrestricted traces sampled CPUs 0, 2, 4, 6, 8, 9, and 10. Fixed-affinity workers stayed within their allowed CPU sets, while unrestricted workers used a wider placement set. This demonstrates the placement effect of processor affinity and the scheduler's greater freedom under unrestricted execution.

More workers did not improve the extension

Extension mean elapsed times were 0.280, 0.380, 0.770, and 1.553 seconds for 1, 2, 4, and 8 workers. Elapsed time increased as worker count increased on the same two CPUs. Each additional worker performed another fixed workload, increasing total work while also increasing competition for the two CPUs.

Limitations and Conclusion

Limitations

1. CPU placement and migration were sampled only 12 times per worker, so migrations between observations were not visible.
2. Only three trials were used per condition, limiting the strength of conclusions about variability.
3. Background processes and operating-system activity may have affected timing.
4. Processor frequency boosting was enabled, so frequency could change during a run.
5. Timing values in timing.csv were rounded to two decimal places, which can hide small differences.
6. The experiment could not observe the scheduler's internal reasons for placement or migration.
7. The extension increased total work as thread count increased, so it was not a fixed-total-work scalability test.

Conclusion

The evidence answered both research questions. CPU affinity and eligible CPU count affected both observed placement and elapsed performance. One-CPU affinity confined all sampled observations to CPU 0 and produced a median elapsed time of 1.130 seconds, compared with 0.280 seconds for unrestricted execution and 0.780 seconds for two-CPU execution. Eight workers on two CPUs produced the slowest common result at 1.540 seconds.

The extension showed that increasing worker count from 1 to 8 on the same two CPUs increased elapsed time from 0.280 to 1.553 seconds. The common-configuration and extension results supported the original predictions. These conclusions apply to the tested Linux environment, workload, CPU sets, sampling procedure, and limited trial count.